

Search and Trees

4/28/25

Administrivia

- To meet with David, sign up on his advising schedule
- HW 6 (Scoring functions for Cartographers) due Wednesday night

Implementing contains (ints)

```
boolean contains(int key) {  
    //returns whether value is in the set  
    for(int i=0; i<size; i++)  
        if(vals[i] == key)  
            return true;  
    return false;  
}
```

Implementing contains (Objects)

```
boolean contains(E key) {  
    //returns whether value is in the set  
    for(int i=0; i<size; i++)  
        if(vals[i].equals(key))  
            return true;  
    return false;  
}
```

Aside: Implementing .equals

```
public class Pair { //represents a pair of ints
    private int x; private int y;
    ...
    public boolean equals(Object o) {
        if(o instanceof Pair) {
            Pair p = (Pair) o;
            return (x == p.x) && (y == p.y);
        }
        return false;
    }
}
```

Recall: Set ADT

- Represents unordered collection of values
 {"hello", "there", "=)", "CS 142"}
 - **No duplicate elements**
 - Supports:
 - boolean add(value) //returns whether it was new
 - boolean contains(value)
 - Iterator iterator()
- Maybe also: size, remove, clear

Dictionary/Map ADT

- `Map<K,V>` //key type K, value type V
 - `V put(K,V)`
 - `V remove(K)`
 - `V get(K)`
 - `boolean contains(K)`
 - `Set<K> keySet()`
 - `Collection<V> values()`
 - `boolean isEmpty()`
 - `int size()`
 - `void clear()`

Array-based implementation of Set

- Store the keys in arbitrary order in an array, keeping the size in another variable
(Basically an ArrayList)

Array-based implementation of Set

- Store the keys in arbitrary order in an array, keeping the size in another variable
(Basically an ArrayList)

Fill in the following: With this implementation, add runs in _____ while contains runs in _____

A. $O(1)$, $O(1)$

B. $O(1)$, $O(n)$

C. $O(n)$, $O(1)$

D. $O(n)$, $O(n)$

E. None of the above

Array-based implementation of Set

- Store the keys in arbitrary order in an array, keeping the size in another variable
(Basically an ArrayList)

Fill in the following: With this implementation, add runs in _____ while contains runs in _____

A. $O(1)$, $O(1)$

B. $O(1)$, $O(n)$

C. $O(n)$, $O(1)$

D. $O(n)$, $O(n)$

E. None of the above

What if the list is kept sorted?

How can you search faster?

Binary search

```
int rank(E key) {    //returns where key goes in keys
    int lo = 0;      //lowest index that could have key
    int hi = size-1; //highest index that could have key
    while(lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = key.compareTo(keys[mid]);
        if(cmp < 0)    hi = mid - 1;
        else if(cmp > 0) lo = mid + 1;
        else return mid;
    }
    return lo;
}
```

Binary search

```
int rank(E key) {    //returns where key goes in keys
    int lo = 0;      //lowest index that could have key
    int hi = size-1; //highest index that could have key
    while(lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = key.compareTo(keys[mid]);
        if(cmp < 0)    hi = mid - 1;
        else if(cmp > 0) lo = mid + 1;
        else return mid;
    }
    return lo;
}
```

How long does this take?

- | | |
|----------------|-------------|
| A) $O(1)$ | D) $O(n^2)$ |
| B) $O(\log n)$ | E) None of |
| C) $O(n)$ | the above |

Binary search

```
int rank(E key) {    //returns where key goes in keys
    int lo = 0;      //lowest index that could have key
    int hi = size-1; //highest index that could have key
    while(lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        int cmp = key.compareTo(keys[mid]);
        if(cmp < 0)    hi = mid - 1;
        else if(cmp > 0) lo = mid + 1;
        else return mid;
    }
    return lo;
}
```

How long does this take?

A) $O(1)$

D) $O(n^2)$

B) $O(\log n)$

E) None of

C) $O(n)$

the above

How long does it take to add a new key
into the sorted array?

- A. $O(1)$
- B. $O(1)$ amortized
- C. $O(\log n)$
- D. $O(n)$
- E. None of the above

How long does it take to add a new key
into the sorted array?

- A. $O(1)$
- B. $O(1)$ amortized
- C. $O(\log n)$
- D. $O(n)$
- E. None of the above

Fill in the following: With a sorted linked list implementation of a set, add runs in _____ while contains runs in _____

- A. $O(1)$, $O(1)$
- B. $O(1)$, $O(\log n)$
- C. $O(\log n)$, $O(1)$
- D. $O(n)$, $O(\log n)$
- E. None of the above

Fill in the following: With a sorted linked list implementation of a set, add runs in _____ while contains runs in _____

- A. $O(1)$, $O(1)$
- B. $O(1)$, $O(\log n)$
- C. $O(\log n)$, $O(1)$
- D. $O(n)$, $O(\log n)$
- E. None of the above: $O(n)$, $O(n)$

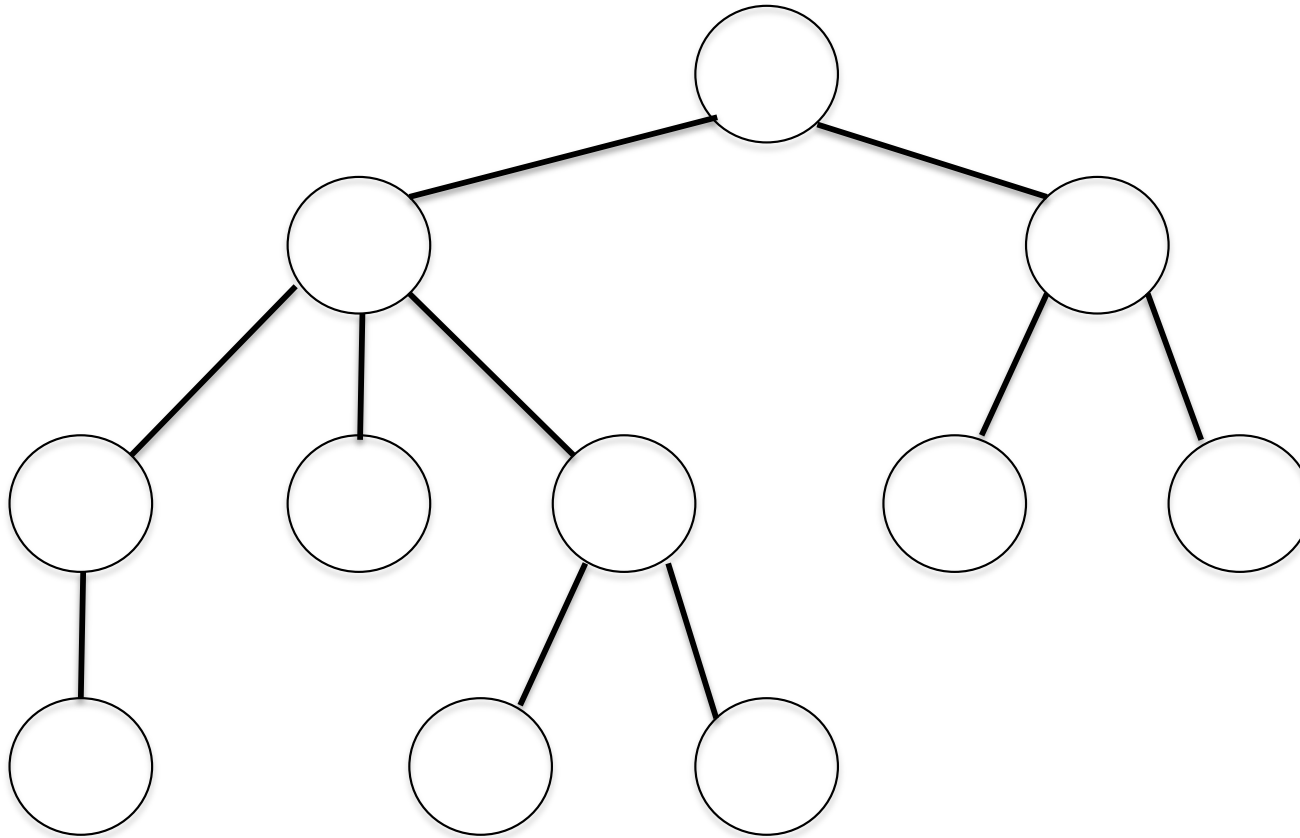
Overview of set implementations

- Array w/ vals in arbitrary order, but no duplicates
 - $O(n)$ add, $O(n)$ contains
- Array with values in sorted order
 - Allows binary search ($O(\log n)$), but slow insertion ($O(n)$)
- Linked list
 - Can more easily add in the middle, but contains is slow

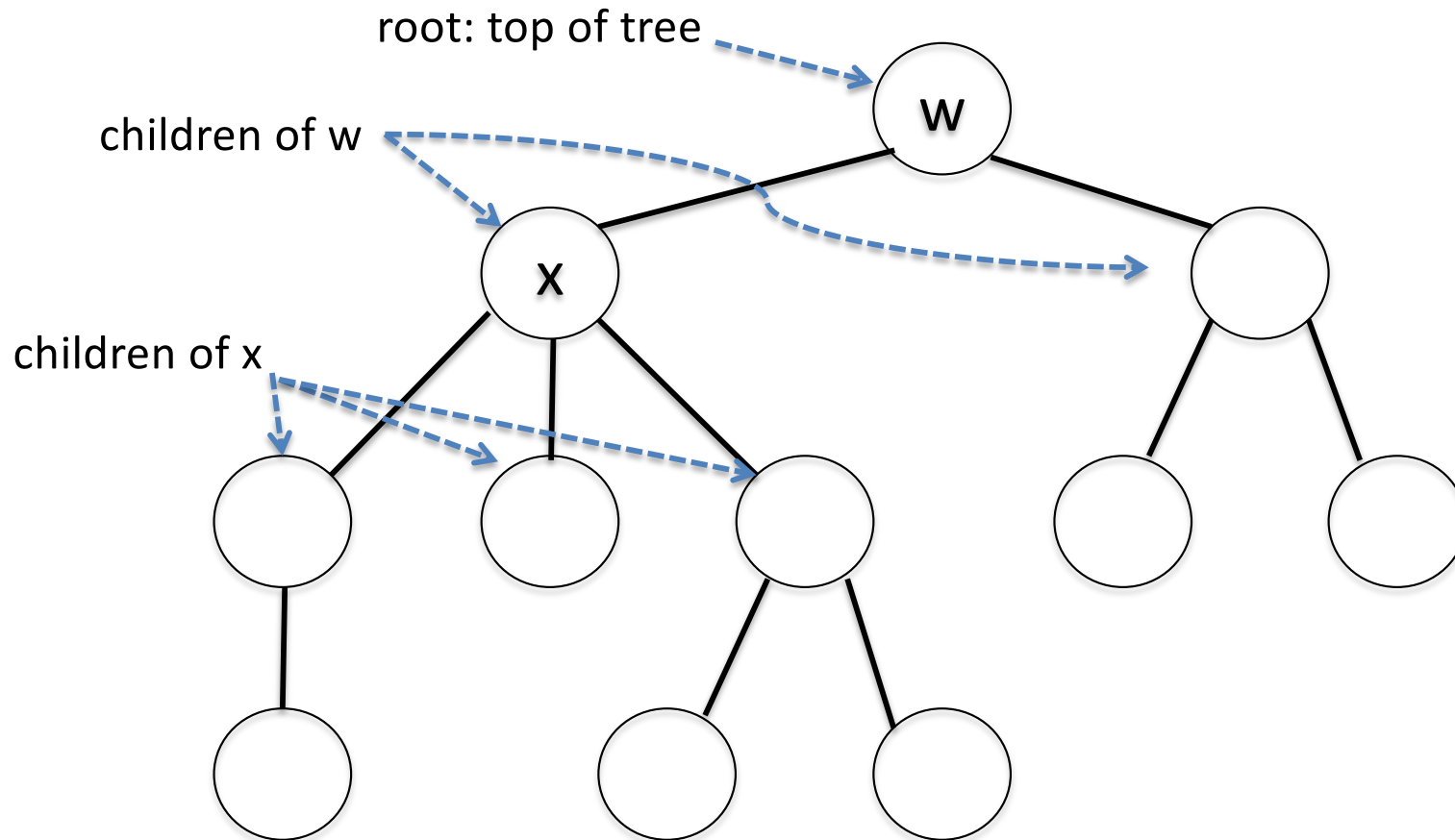
Desired properties

- Ability to quickly “jump” later in the sequence of keys (like binary search)
- Ability to easily add keys into the appropriate position (like linked lists)

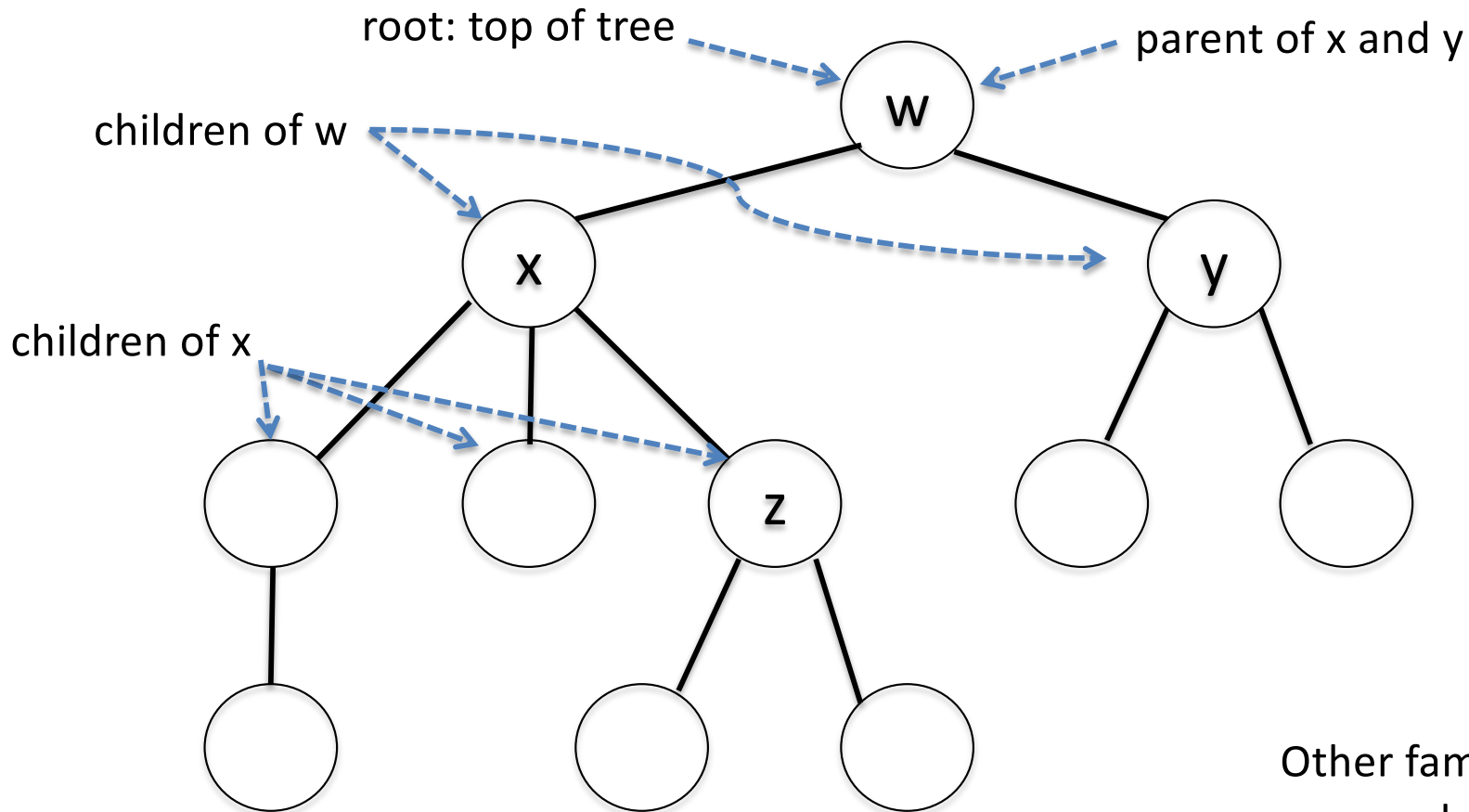
Trees



Trees



Trees



Other familial relationships:

x and y are siblings

w is grandparent of z

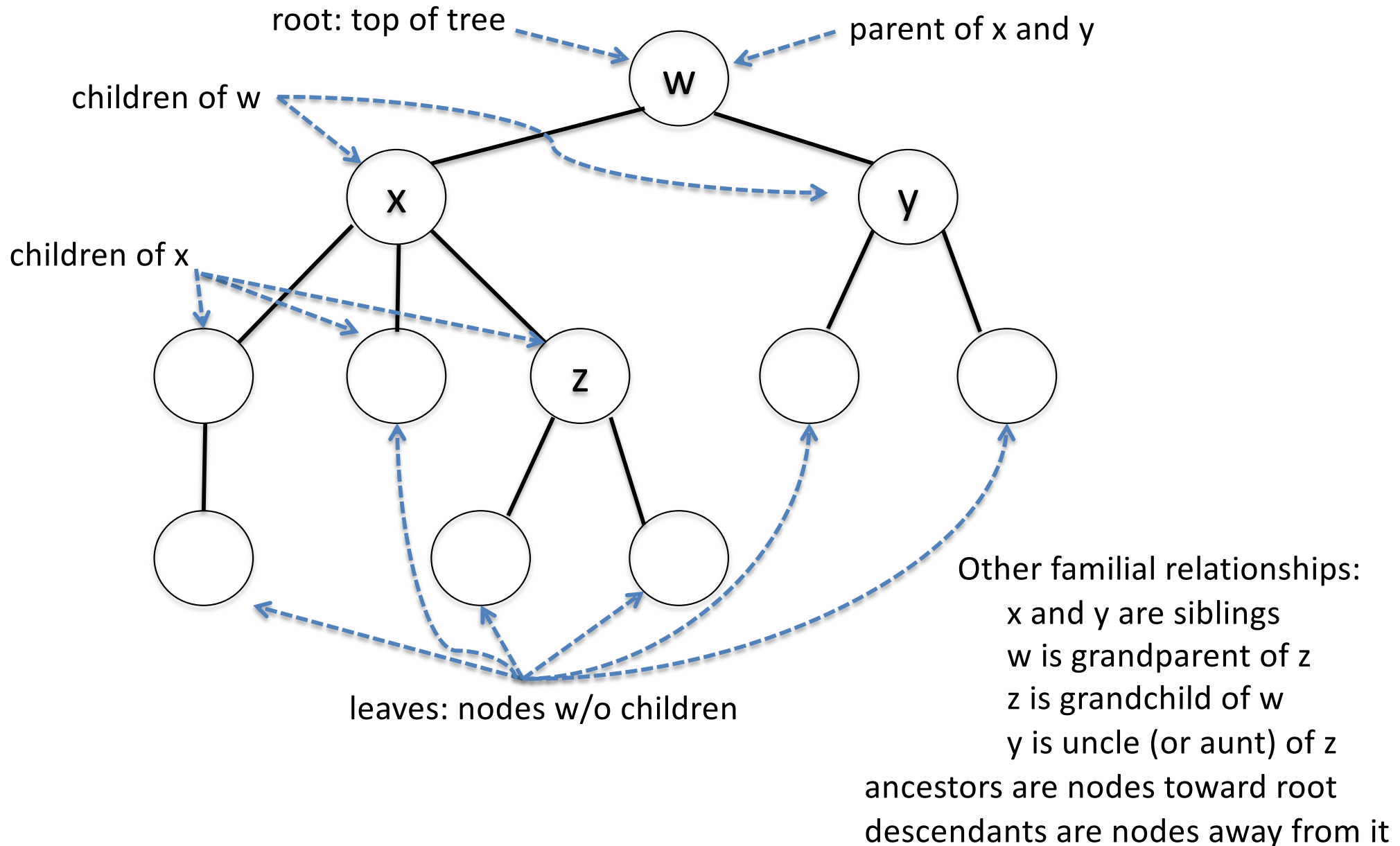
z is grandchild of w

y is uncle (or aunt) of z

ancestors are nodes toward root

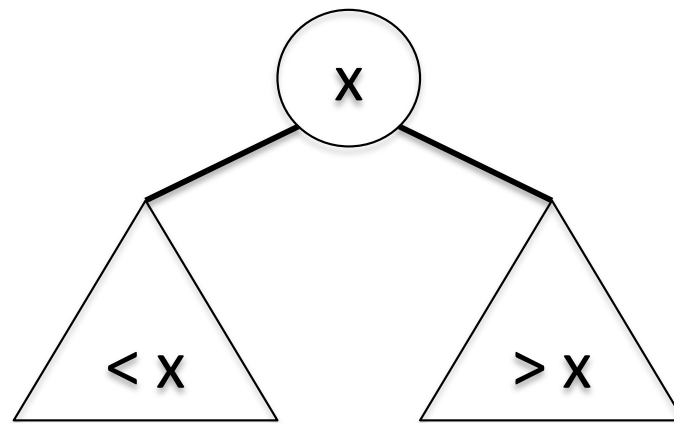
descendants are nodes away from it

Trees

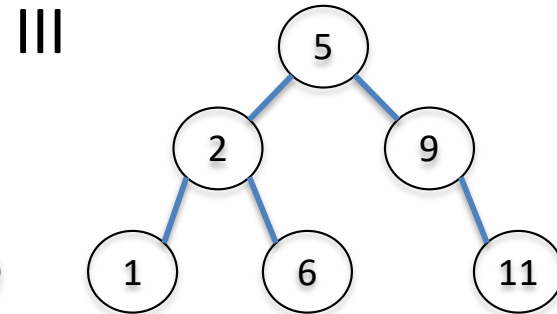
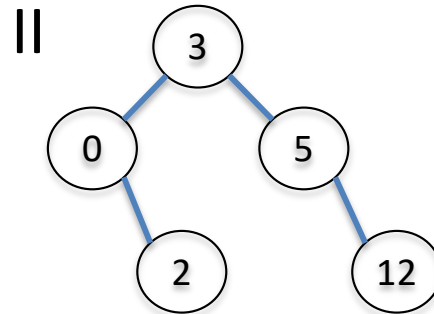
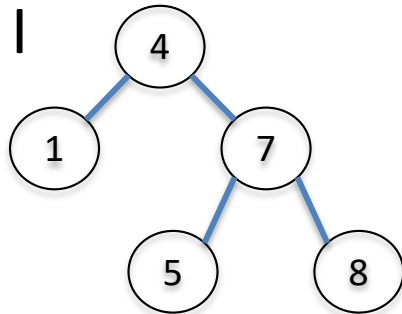


Binary Search Trees (BSTs)

- Tree with key stored at each node such that
 - Every node has 2 children (left and right)
 - Children can be null (nodes have 0, 1, 2 actual children)
 - All keys in a node's right subtree are greater than its key and all in the left subtree are less

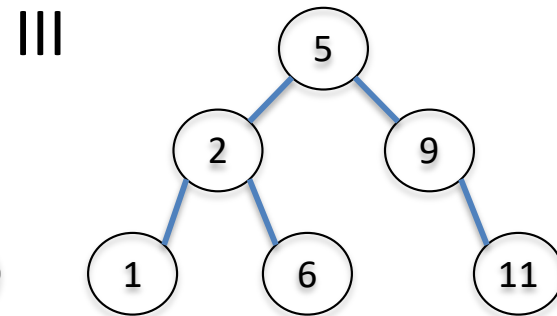
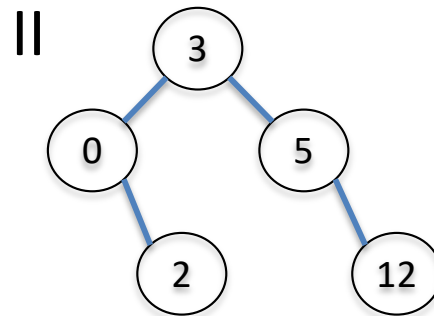
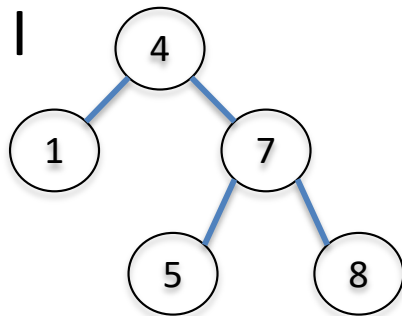


Which of the following is a legal BST?



- A. I only
- B. I and II only
- C. II and III only
- D. All of them
- E. None of the choices above is correct

Which of the following is a legal BST?



A. I only

B. I and II only

C. II and III only

D. All of them

E. None of the choices above is correct

Implementing a BST

```
public class BST<E extends Comparable<E>> {  
    Node root;  
  
    private class Node {  
        Node left;  
        Node right;  
        E key;  
    }  
}
```

How can a BST implement contains?

How can a BST implement contains?

```
boolean contains(E key) {  
    Node curr = root;  
    while(curr != null) {  
        int cmp = key.compareTo(curr.key) ;  
        if(cmp < 0) curr = curr.left;  
        else if(cmp > 0) curr = curr.right;  
        else return true;  
    }  
    return false;  
}
```